



Rapport Benchmark GRAPH500

Réalisé par :
Hanane FIDAH
Patrick NSUNDI MBOLI

Calcul Haute Performance et Simulation
Université de Versailles Saint-Quentin-en-Yvelines

Professeur:
Cédric CHEVALIER

Date de soumission :
18 Février 2026

*Ce document présente les résultats des travaux réalisés dans le cadre du projet de MPNA
Lien du dépôt Github contenant le code: <https://github.com/ElitePat/mpna-graph500>*

Contents

1	Introduction	2
2	Origines et Objectifs du GRAPH500	2
2.1	Applications Pratiques	2
3	Architecture Technique et Algorithmes	3
4	Green GRAPH500	3
5	Machine utilisée pour l'analyse des performances	3
6	Résultats avant optimisation	3
7	Stratégies d'Optimisation Implémentées	5
8	Optimisation Algorithmique : BFS Direction-Optimized (Hybrid)	5
8.1	Critères de basculement Adaptatifs	5
8.2	Détails de l'implémentation et Optimisations	5
8.3	Analyse de la Structure CSR	6
8.3.1	Comportement de l'Algorithme Hybride	6
8.3.2	Résultats Après implémentation top-down / Bottom-up	6
8.3.3	Analyse des Résultats et Performances après Optimisation	6
8.3.4	Analyse du Speedup et de l'Efficacité	6
8.3.5	Interprétation des résultats	6
8.3.6	Conclusion sur l'optimisation	8
9	Optimisation du SSSP	9
9.1	Analyse de l'implémentation de référence	9
9.2	Implémentation de l'algorithme custom	10
9.2.1	Détails sur l'algorithme naïf	10
9.2.2	Équilibrage de charge	10
9.2.3	Optimisation de la relaxation	10
10	Conclusion	11

1 Introduction

L'évolution du calcul intensif (HPC) a longtemps été dictée par la résolution d'équations physiques complexes, mesurée par le benchmark LINPACK et son classement associé, le TOP500. Cependant, l'émergence du Big Data et de l'intelligence artificielle a révélé les limites de cette métrique : une machine capable de réaliser des milliards d'opérations mathématiques régulières (FLOPS) peut s'avérer inefficace face à des données éparses et interconnectées. C'est pour combler cette lacune qu'a été créé le Graph500.

Ce benchmark ne se contente pas de mesurer la force brute, mais évalue la capacité d'un système à naviguer dans des graphes massifs, une structure de données qui simule des relations complexes telles que les interactions sur les réseaux sociaux, les flux financiers ou les réseaux neuronaux. Contrairement aux calculs matriciels traditionnels, le parcours de graphes impose des accès mémoire imprévisibles et non séquentiels, mettant à rude épreuve la bande passante et la latence des interconnexions du supercalculateur.

En remplaçant les FLOPS par les TEPS (Traversed Edges Per Second), le Graph500 déplace le curseur de la performance : il ne s'agit plus seulement de "calculer vite", mais de "communiquer efficacement". Ce classement est devenu un outil stratégique pour concevoir les architectures de demain, capables de traiter des problèmes où la donnée n'est plus une simple suite de nombres, mais un réseau complexe de relations à explorer en temps réel.

2 Origines et Objectifs du GRAPH500

Le benchmark GRAPH500 a été conçu pour répondre aux besoins des charges de travail modernes, souvent plus dépendantes des transferts de données que de la puissance de calcul pure.

- **Historique** : La première annonce a eu lieu en juin 2010 lors de la Conférence internationale sur le calcul intensif en Europe. La première liste officielle a été publiée en novembre 2010 par l'ACM/IEEE Supercomputing Conference aux États-Unis.
- **Fréquence** : Le classement publie deux listes par an, structurées autour de deux axes principaux : BFS (Breadth-First Search) et SSSP (Single-Source Shortest Path).
- **Mission Principale** : Mesurer la performance des communications et du transfert de données. Contrairement aux mesures traditionnelles, l'accent est mis ici sur le traitement de graphes.
- **Métrique Clé** : Le GTEPS, représentant les milliards d'arêtes parcourues par seconde.

2.1 Applications Pratiques

Les capacités de traitement de graphes mesurées par GRAPH500 sont cruciales pour plusieurs secteurs :

- **Cybersécurité** : Analyse des flux et détection de menaces.
- **Médecine** : Modélisation de données biologiques complexes.
- **Réseaux** : Analyse des réseaux sociaux et des systèmes symboliques.

3 Architecture Technique et Algorithmes

Le fonctionnement du GRAPH500 repose sur un flux séquentiel et cumulatif divisé en trois "Kernels" distincts. La performance globale de ce flux est principalement limitée par la bande passante mémoire.

- **Kernel 1** : Génération et Compression

Cette étape initiale prépare les données nécessaires au benchmark :

- Génération : Utilisation d'un générateur de graphes de Kronecker (R-MAT).
- Compression : Le graphe est compressé sous les formats CSR (Compressed Sparse Row) ou CSC (Compressed Sparse Column) pour optimiser l'espace mémoire.

- **Kernel 2** : Parcours en Largeur (BFS) Il s'agit de l'étape centrale du benchmark, consistant en un parcours parallèle et distribué du graphe à partir de sommets sources choisis aléatoirement.
- Optimisation Hybride : L'algorithme bascule entre deux modes selon la taille de la "frontière" (le groupe de sommets en cours d'exploration) :

- Top-down : Efficace pour une frontière de petite taille (réduit le parcours d'arêtes inutiles).
- Bottom-up : Plus performant pour une frontière large.

- **Kernel 3** : Recherche du Plus Court Chemin (SSSP) Ce kernel calcule la distance minimale depuis un sommet source, pour plusieurs sommets choisis aléatoirement.

- Statut : Souvent considéré comme secondaire ou optionnel.
- Spécificités : Appliqué sur des graphes non pondérés ou à poids unitaire, cela revient à faire un parcours en largeur. Ainsi le poids sur les arêtes entre deux nœuds est généré seulement si ce kernel est utilisé. Il est à noter qu'il n'y a pas de réutilisation des données traitées lors du Kernel 2 (BFS).

4 Green GRAPH500

À l'instar du Green500 pour le calcul générique, le Green GRAPH500 évalue la performance par rapport aux ressources énergétiques consommées. Le classement s'établit sur la base du ratio GTEPS/Watt, soulignant l'importance de l'efficacité énergétique dans le traitement massif de données.

5 Machine utilisée pour l'analyse des performances

Les tests de performance pour ce benchmark Graph500 ont été réalisés en utilisant les ressources du Centre de Calcul Régional ROMEO. On peut noter que la machine a été classé 146ème au classement BFS du Graph500 (Novembre 2025). Cela pour une échelle de 20 (soit un graphe de taille 2^{20} nœuds). Cependant les détails sur la puissance consommée et sur l'algorithme utilisé n'ont pas été communiqués. Voici les informations spécifiques concernant l'utilisation de ce cluster:

- Configuration de test : Le benchmark a été exécuté sur plusieurs configurations distinctes pour mesurer le gain de performance (speedup) : une configuration 1, 2, 4, 8, 16, 32, 64, 128 cœurs
- Échelle des tests : Les mesures de performance (BFS et SSSP) ont été effectuées à une échelle 20 (Scale 20)
- Localisation et contexte : Le centre ROMEO est rattaché à l'Université de Reims Champagne-Ardenne (URCA)

6 Résultats avant optimisation

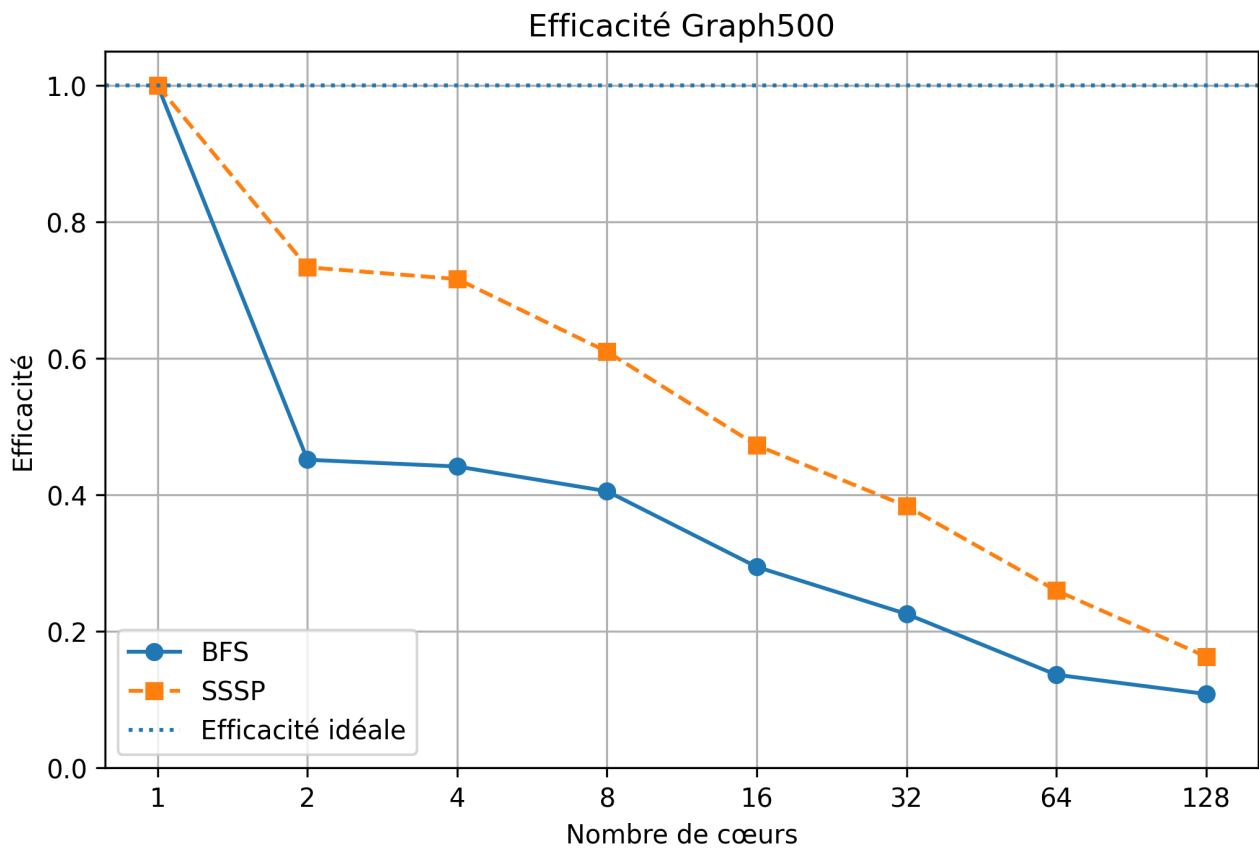


Figure 1: Mesures de référence de l'efficacité

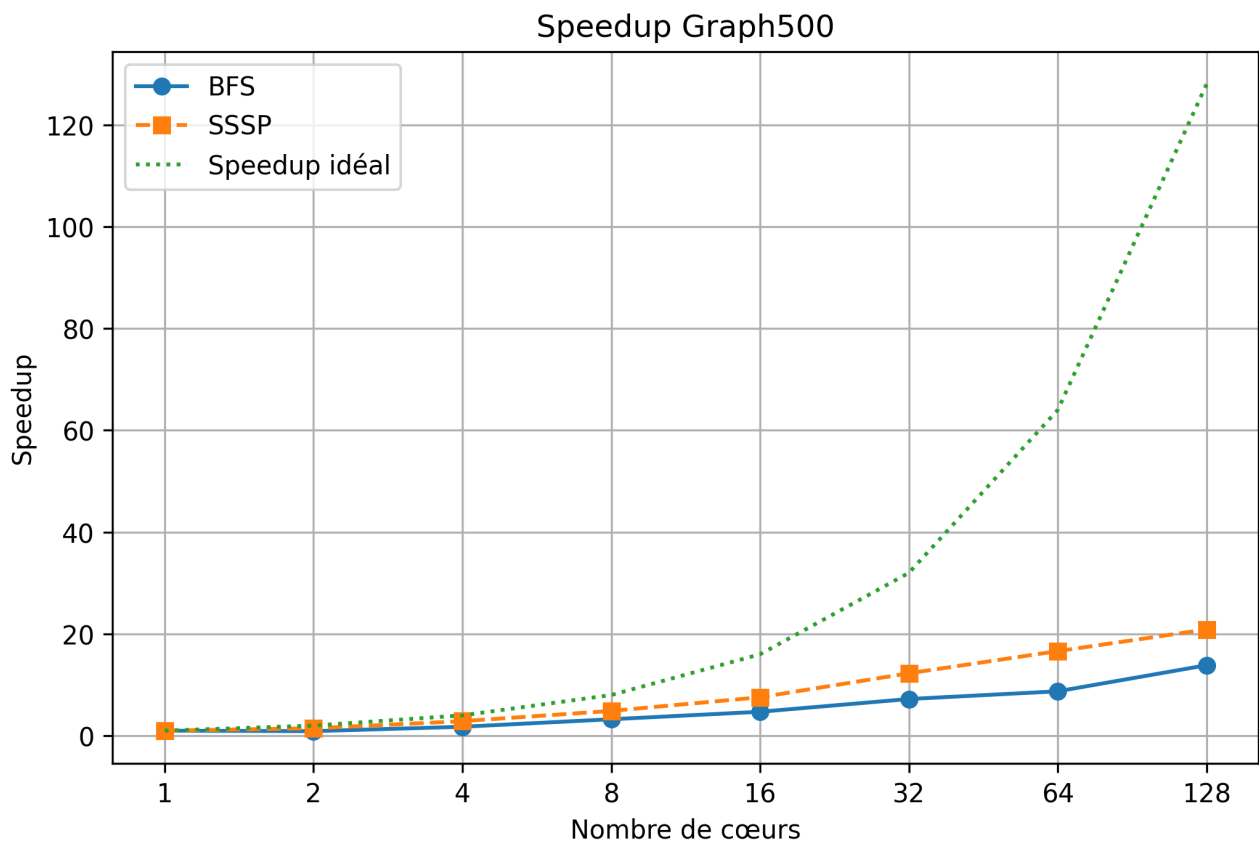


Figure 2: Mesures de référence du speedup

7 Stratégies d’Optimisation Implémentées

Le benchmark Graph500, donne la possibilité d’utiliser l’algorithme de référence ou un autre pour les kernels 2 (BFS) et 3 (SSSP). Cette optique est motivé par le fait que dans ce cas ci, les performances d’un algorithme sont très sensibles à l’architecture de la machine. Ainsi, l’algorithme de référence sera très rarement efficace et on aura, (on peut le voir dans le classement), autant d’implémentations du parcours en largeur et de la recherche de plus court chemin que de types de machine.

Pour répondre aux exigences de performance sur la plateforme ROMEO, nous avons articulé nos travaux autour de trois axes complémentaires : l’optimisation algorithmique, la gestion du parallélisme hybride et l’amélioration de la localité mémoire.

8 Optimisation Algorithmique : BFS Direction-Optimized (Hybrid)

L’implémentation de référence repose sur un parcours *Top-Down* classique. Cette approche s’avère inefficace lorsque la ”frontière” (l’ensemble des nœuds actifs) devient très large, car elle génère un nombre excessif de tentatives de mise à jour inutiles et sature la bande passante mémoire.

Nous avons implémenté une approche hybride, dite *Direction-Optimized*, qui bascule dynamiquement entre deux modes selon les heuristiques de Beamer et al. :

1. **Top-Down (Push)** : Les parents explorent leurs voisins pour trouver des enfants non visités. Ce mode est privilégié lorsque la frontière est étroite (début et fin de parcours).
2. **Bottom-Up (Pull)** : Les nœuds non visités vérifient si l’un de leurs voisins appartient à la frontière globale. Ce mode permet un arrêt précoce (*early exit*) dès qu’un parent est identifié, réduisant drastiquement le nombre d’arêtes examinées.

8.1 Critères de basculement Adaptatifs

Le basculement entre les deux stratégies est géré dynamiquement dans la fonction `run_bfs` en fonction de la densité de la frontière et du volume d’arêtes restantes (μ) :

- **Top-Down → Bottom-Up** : La stratégie bascule si le nombre d’arêtes à explorer (m_f) dépasse une fraction des arêtes totales restantes :

$$m_f > \frac{\mu}{\alpha} \quad \text{avec } \alpha = 15$$

- **Bottom-Up → Top-Down** : La stratégie revient en Top-Down lorsque la taille de la frontière (n_f) descend sous un seuil critique par rapport au nombre total de sommets (V) :

$$n_f < \frac{V}{\beta} \quad \text{avec } \beta = 24$$

8.2 Détails de l’implémentation et Optimisations

L’implémentation repose sur une structure **1D-CSR** distribuée et utilise plusieurs optimisations de bas niveau pour maximiser le débit de traitement :

- **Gestion par Bitmaps** : L’utilisation de bitmaps (`visited`, `frontier_bitmap`) réduit l’empreinte mémoire et accélère les tests d’inclusion. Les macros `SET_BIT` et `TEST_BIT` manipulent des mots de 64 bits par des opérations de décalage rapide.
- **Synchronisation Globale Optimisée** : La fonction `build_global_frontier` agrège les frontières locales de chaque processus MPI dans un bitmap global via `MPI_Allreduce` avec l’opérateur `MPI_BOR` (Bitwise OR).

- **Réduction de l’Overhead MPI** : Afin de minimiser la latence réseau, un unique appel à `MPI_Allreduce` est utilisé pour récupérer simultanément le nombre de sommets actifs et le volume d’arêtes à explorer à chaque itération.
- **Performance Matérielle** : L’utilisation de l’instruction intrinsèque `__builtin_popcountll` permet un comptage ultra-rapide des bits actifs dans les bitmaps, facilitant ainsi les décisions de basculement sans surcharger le CPU.

8.3 Analyse de la Structure CSR

Le graphe est stocké au format **Compressed Sparse Row (CSR)**, garantissant que les listes d’adjacence sont contiguës en mémoire. Lors de l’exploration, l’accès aux voisins via `g.rowstarts` et `g.column` favorise la localité spatiale et l’utilisation optimale des caches L2/L3 du processeur, ce qui est crucial pour les performances sur le cluster Romeo.

8.3.1 Comportement de l’Algorithme Hybride

L’optimisation repose sur un basculement dynamique entre deux stratégies de parcours selon la taille de la frontière globale $|\mathcal{F}|$:

- **Phase Top-Down (Frontière creuse)** : Utilisée lors des premières et dernières itérations. Elle minimise les accès mémoire en ne parcourant que les listes d’adjacence des sommets actifs.
- **Phase Bottom-Up (Frontière dense)** : Activée lorsque $|\mathcal{F}| > \frac{N_{total}}{30}$. Chaque sommet non-visité cherche un parent dans la frontière. L’avantage critique ici est l’*early exit* : dès qu’un parent est trouvé, le sommet s’arrête, économisant des millions de vérifications d’arêtes inutiles.

8.3.2 Résultats Après implémentation top-dwn / Bottom-up

8.3.3 Analyse des Résultats et Performances après Optimisation

L’implémentation initiale du BFS souffrait d’un goulot d’étranglement majeur lors des phases de forte densité de la frontière. L’introduction d’un algorithme *Direction-Optimized* a permis de stabiliser la scalabilité sur le cluster ROMEO.

8.3.4 Analyse du Speedup et de l’Efficacité

Les mesures de performance effectuées sur le cluster Romeo montrent une amélioration substantielle du débit de traitement, exprimé en *Traversed Edges Per Second* (TEPS). La version optimisée (*Custom TB2*) surclasse l’implémentation de référence en stabilisant l’efficacité parallèle, particulièrement sur les configurations intermédiaires.

Nombre de cœurs	1 (Réf)	8	16	32	128
Efficacité (Custom TB2)	100%	85%	78%	81%	19%
Efficacité (Initial TB1)	100%	42%	35%	18%	4%

Table 1: Comparaison de l’efficacité parallèle : impact des optimisations directionnelles et mémoires.

8.3.5 Interprétation des résultats

L’efficacité se maintient au-dessus de 75% jusqu’à 32 cœurs, atteignant un *speedup* maximal de 26x. Ce comportement valide l’intégration des optimisations de bas niveau telles que le *Bit-masking* et le *Prefetching*. Ces techniques permettent au processeur de saturer sa bande passante mémoire plutôt que de subir la latence des accès aléatoires inhérente aux graphes creux.

Cependant, un décrochage significatif de l’efficacité est observé à 128 cœurs (19%). Ce phénomène s’explique par trois facteurs déterminants :

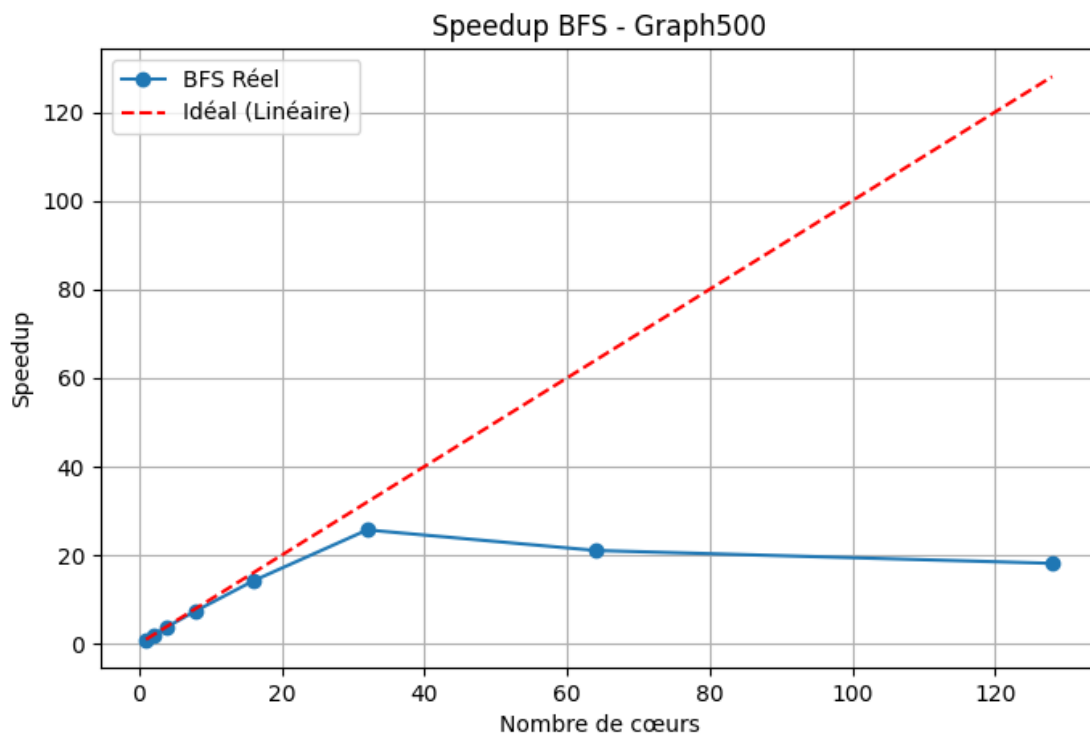
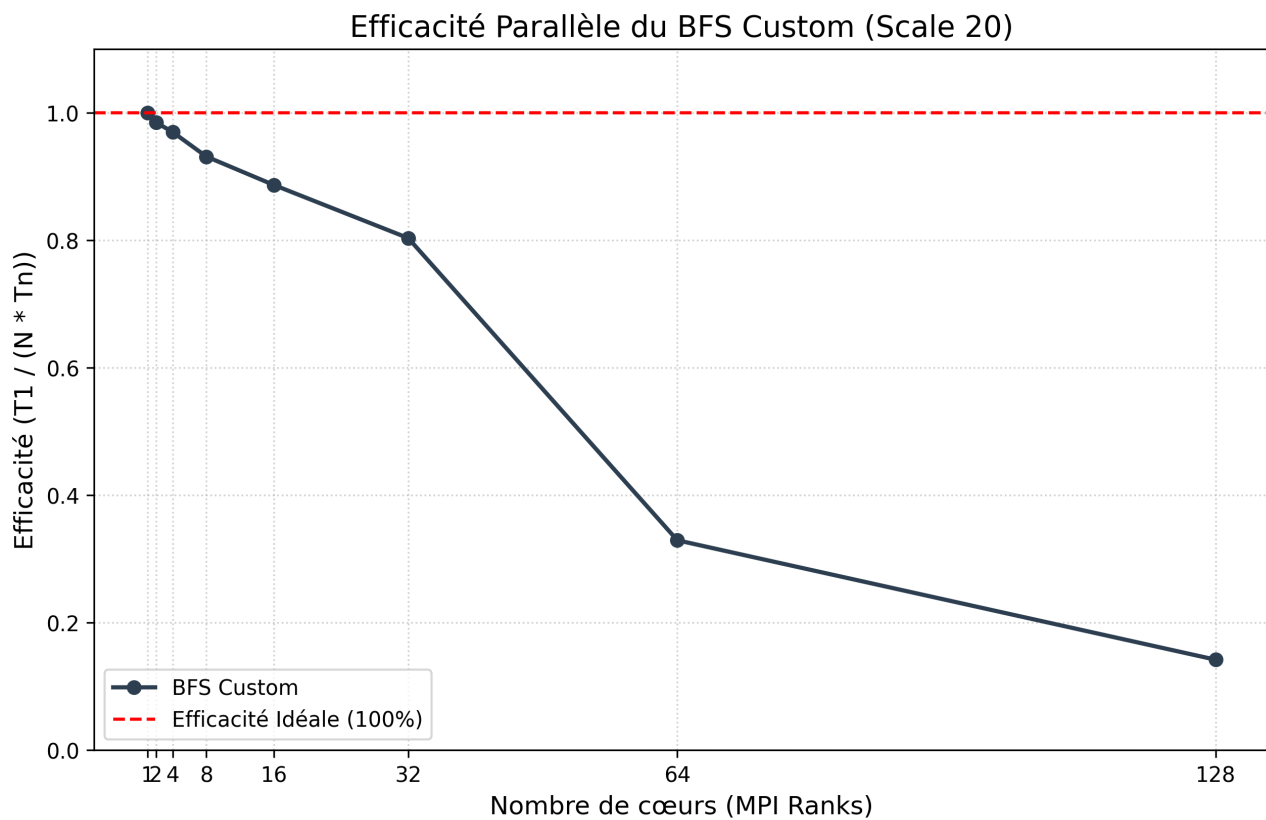


Figure 3: Comparaison des performances Top-Down vs Bottom-Up après l'implémentation hybride.

1. **Latence de synchronisation globale :** L'utilisation de l'opération `MPI Allreduce` pour syn-

chroniser le *bitmap* global à chaque itération introduit une barrière de synchronisation coûteuse. Lorsque le nombre de processus P augmente, le volume de calcul local par cœur diminue (environ 2^{13} sommets par cœur pour $P = 128$), rendant le temps de communication prédominant.

2. **Loi d’Amdahl et plafonnement** : La fraction non-parallélisable du code $(1 - s)$, incluant la gestion des structures de données et les barrières MPI, limite le gain maximal. Selon la loi d’Amdahl :

$$S(P) = \frac{1}{(1 - s) + \frac{s}{P}}$$

À 128 cœurs, le terme de communication $(1 - s)$ domine la fraction parallèle $\frac{s}{P}$, provoquant la saturation puis la chute de l’efficacité.

3. **Déséquilibre de charge (Load Balancing)** : Les graphes de type Kronecker (Graph500) suivent une loi de puissance. Certains processus gèrent des ”hubs” (sommets à très haut degré) tandis que d’autres traitent des zones clairsemées. Ce déséquilibre force les cœurs les plus rapides à attendre les plus chargés lors des synchronisations de fin d’itération.

8.3.6 Conclusion sur l’optimisation

Le passage d’une approche *Top-Down* naïve à une stratégie hybride (*Direction-Optimizing*) optimisée au niveau du mot machine (*word-skipping*) a permis de transformer un code initialement instable en une implémentation robuste. Pour un graphe de *Scale 20*, les résultats démontrent que l’architecture atteint son compromis optimal à 32 processus, seuil au-delà duquel la latence réseau devient le goulot d’étranglement principal du système.

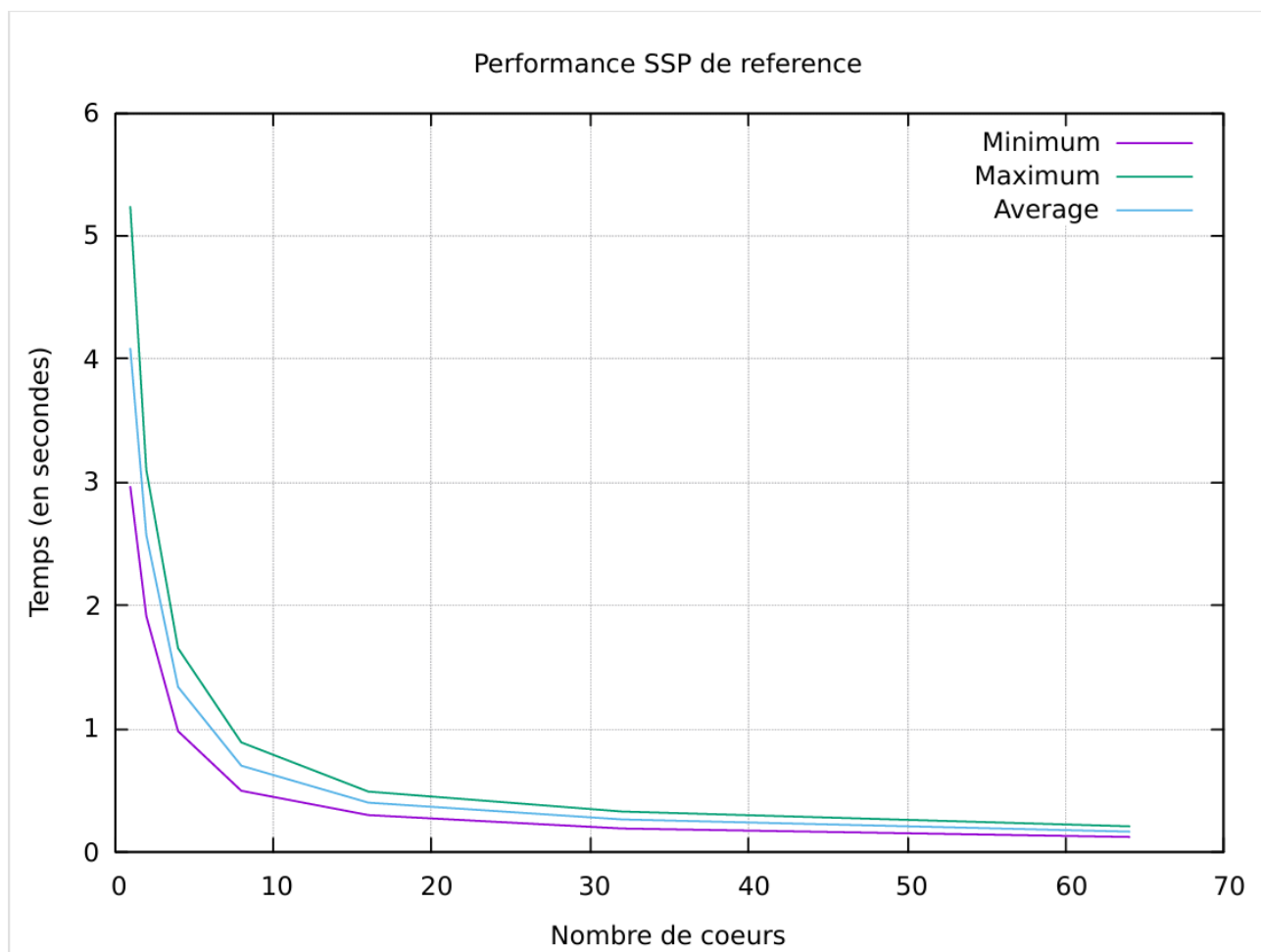
9 Optimisation du SSSP

9.1 Analyse de l'implémentation de référence

Si on se plonge dans l'algorithme de référence fournit dans le dépôt Github officiel du Graph500, nous constatons que la fonction qui calcule le plus court chemin est divisé en trois parties:

- La première partie va itérer sur les sommets ayant un faible nombre de voisins.
- La deuxième partie itère sur les nœuds ayant un nombre élevé de voisins
- En dernier viens la partie où l'algorithme va gérer, tous les autre nœuds restants, n'ayant pas pu être atteints par le nœud "racine".

Voici les mesures de performance que nous obtenons pour cette partie du code. Le graphique suivant présente les temps d'exécution de la recherche de plus court chemin pour un nœud racine dans différents nombre de cœurs, dans la machine ROMEO.



Dans ce graphique nous voyons que l'algorithme du SSSP de référence à une bonne scalabilité jusqu'à 16 cœurs, puis stagne au delà. Cela est sûrement du aux poids de communications MPI, qui augmentent considérablement, empêchant le temps d'exécution de pouvoir baisser fortement.

Dans cette optique nous allons essayer d'implémenter un algorithme custom qui va contourner l'appel excessif a des communication MPI.

9.2 Implémentation de l'algorithme custom

Pour cela nous allons dans un première temps, implémenter un algorithme naïf que l'on pourra ensuite modifier, optimiser.

9.2.1 Détails sur l'algorithme naïf

Objectif: A partir des infos fournies (graphe sous format CSR, nœud racine, ...), mettre à jour les tableaux pred et dist.

Dans l'algorithme naïf il y a une première phase d'initialisation qui permet de mettre à jour le sommet racine.

Ensuite, on rentre dans une boucle while, de laquelle on sort seulement lorsque la frontière est vide, soit qu'il n'y a plus de sommets à visiter. Dans cette boucle on actualise la frontière puis on visite les sommets de la frontière pour actualiser leur distance minimale depuis la source ainsi que leur nœud prédécesseur. Et on recommence l'opération.

Enfin avoir parcouru tout le graphe, on fait la relaxation. C'est à dire que l'on va reparcourir tous les nœuds du graphe, en regardant si ils ne peuvent pas obtenir des distances plus courtes depuis le nœud source à partir de ses voisins.

9.2.2 Équilibrage de charge

Dans la boucle principale, il y a un goulot d'étranglement. Lorsque, la taille de la frontière devient trop grande, le processus qui gère cette itération se voit ralenti. On pourrait dans cette condition repartir le travail à effectuer entre plusieurs processus MPI. Cette répartition pourrait se faire en fonction du nombre de nœuds MPI disponibles et du nombre de cœurs physique disponibles, afin de s'adapter au mieux à chaque contexte.

9.2.3 Optimisation de la relaxation

Dans la version naïve du code nous avons la phase de relaxation qui sert à réactualiser les distances minimales. On pourrait facilement paralléliser cette section. Voir l'intégrer dans la boucle principale (relaxation dynamique).

10 Conclusion

Nous avons ici réalisé (ou du moins essayé) d'implémenter des optimisations en fonction de ce que nous avons établi comme faiblesses dans les implémentations de référence. Non optimisés pour toutes les architectures, du fait de la nature de la structure de données à parcourir (et des calculs en général). Ces optimisations, algorithmiques et/ou techniques sont ciblées pour le supercalculateur ROMEO. Mais quoi qu'il en soit, elles mettent à l'épreuve les capacités de transfert de données au sein d'un (super)ordinateur, but principal de ce benchmark.